Combined Revision

VLSI Design Flow: RTL to GDS | Weeks 1 to 3

One file for the prepared revision guides. Every weekly explanation, worked example, handwritten page and source link is retained. Choose a week below or use the PDF bookmarks.

Week	Revision guide	PDF pages
01	IC foundations and pre-RTL design	2-9
02	Implementation, verification, manufacturing and Tcl	10-22
03	Verilog, simulation and high-level synthesis	23-38

38 pages in this edition. Page numbers in the right-hand column and the lower-right footer refer to this combined PDF. The contents link at the left of each weekly footer returns to that week; **All weeks** returns here.

When another week is prepared, it will be added to this same file. The download filename stays **Combined Revision.pdf**; download it again to get the expanded edition.

Coverage includes only the weeks listed above. The course has 12 weeks; later guides are added when requested. The local app remains for assignment practice.

Week 1 - Revision

IC foundations, design choices and pre-RTL methods

Four lectures plus Tutorial 1. The first logic-synthesis lecture belongs to Week 2, even though it appears in the existing Day 01 reading edition. This short guide follows the course week. [1]

1. ICs | 2. Styles | 3. Economics | 4. Partitioning | 5. HLS | 6. Trade-offs | 7. Numericals | 8. Unix/sources

1. IC structure and fabrication

An **integrated circuit (IC)** combines devices and interconnections on a semiconductor die. Silicon forms the substrate and device regions in conventional CMOS. Metals provide interconnect; dielectric separates conductors; a **via** joins selected metal layers. Two wires crossing in a top view need not connect if they occupy insulated layers with no via. More routing layers provide freedom, with resistance, capacitance and process rules still present. [2]

Term	Meaning and distinction
Ingot	Bulk crystal before thin wafers are sliced
Wafer	Thin substrate carrying many die sites
Die	One circuit region, later separated by dicing
Packaged chip	Die connected and protected for use in a system
Photomask	Pattern used during an exposure step
Photoresist	Temporary light-sensitive coating; not the final metal

Simplified pattern transfer: deposit the film, coat resist, expose through a mask, develop the resist, etch the exposed film and strip the resist. Lithography defines the resist pattern; etching transfers it. Different patterned layers normally need different masks and accurate alignment. [2]

Remember: Moore's observation concerns a historical growth trend in device integration. A doubling in transistor count does not by itself guarantee doubled clock frequency, halved energy or doubled application speed.

2. Implementation styles and figures of merit

Style	What is customized	Advantage	Limitation
Full custom	Device choices and detailed physical design	Greatest freedom for demanding blocks	High effort, verification cost and turnaround
Standard-cell ASIC	Network built from characterized cells	Automation with efficient application-specific hardware	Up-front design/mask cost; fabrication commitment
Traditional gate array	Mainly selected interconnect layers over a prepared base	Reduced customization effort in its intended flow	Less freedom than a fully customized implementation
FPGA	Configuration of prefabricated programmable resources	Fast implementation and field updates	Programmability overhead in unit cost, area, power or timing

These are implementation styles. **Fabless**, **foundry** and **integrated device manufacturer (IDM)** describe business responsibilities. A fabless company designs; a foundry fabricates; an IDM combines design and manufacturing roles. A **PDK** supplies process-related models, rules and data needed to design for a technology. A PDK is not the customer's functional specification. [2,3]

Power, performance and area (PPA) are competing measures. Specify performance as latency, throughput or frequency. Specify area as cell, core or die area. A smaller sum of cell areas does not alone prove a smaller routable die. Testability, reliability, cost and time to market can also decide the implementation. [3]

Advantage/disadvantage questions: identify the workload and production volume first. An FPGA may fit a changing low-volume product; a high-volume ASIC may recover a larger fixed investment through lower cost per unit. Neither choice is universally best.

3. Economics and numerical checkpoints

For a stated linear model, total cost is fixed development cost plus quantity times unit cost:

$$C = F + Nv, \qquad N_{BE} = \frac{F_A - F_B}{v_B - v_A}.$$

The graph intercept is fixed cost; its slope is unit cost. A positive crossover needs a meaningful positive quantity. Equal unit costs do not recover a higher fixed investment. These are teaching models, not current commercial quotations. [3]

Worked example: ASIC cost is 3,600,000 + 18N and FPGA cost is 600,000 + 78N. Fixed-cost difference is 3,000,000; saving per unit is 60. The crossover is **50,000 units**. Below it, FPGA cost is lower in this model; above it, ASIC cost is lower. Do not divide by either unit cost alone.

Yield connects manufacturing to unit cost. If a wafer costs 8,000, contains 400 gross dies and has 80% die yield, it gives 320 good dies. Wafer cost per good die is 25. If packaging and final test add 6 per good die, with no further losses, finished-unit cost is 31. The denominator is good dies, not all die sites.

Quantity	Calculation	Common error
Die yield	Good dies / gross dies	Inverting the fraction
Cost per good die	Wafer cost / good dies	Ignoring yield
Task energy	Average power times duration	Treating lower power as automatically lower energy
Frequency	Reciprocal of clock period	Mixing ns and ps

For the capacitive switching component, $P = \alpha C V^2 f$ when α counts average zero-to-one charging events per cycle. A voltage reduction to 0.8 of its original value gives a 0.64 power ratio if activity, capacitance and feasible frequency stay unchanged. Leakage and short-circuit power are separate effects.

4. Abstraction, partitioning and IP integration

Abstraction controls how much implementation detail a model exposes. An algorithm says what to compute; RTL makes register transfers and clocked behavior explicit; a gate netlist gives logical structure; layout adds physical geometry. Lowering abstraction usually adds detail and makes exploration more expensive. A high-level estimate is useful, but it is not final physical signoff. [4]

Hardware-software partitioning assigns functions to dedicated hardware or software on a processor. Hardware can exploit parallelism and tailored datapaths; software offers flexibility and easier updates. Consider acceleration benefit, transfer overhead, memory access, power, area, cost and engineering effort together.

Worked example: software takes 80 ms. An accelerator needs 8 ms computation plus 12 ms communication and coordination. The system now takes 20 ms, so speedup is 4, rather than the compute-only factor of 10. Accelerate the measured bottleneck and include interface costs.

A **system-on-chip (SoC)** integrates interacting processing, memory, communication and other functions. **Intellectual property (IP)** here means reusable design content, not a guarantee that every integration is correct. Soft IP is commonly supplied as synthesizable RTL; hard IP has a process-specific physical implementation. [5]

Reuse benefit	Integration obligation
Avoid rebuilding an established block	Check its configuration and documented operating assumptions
Reuse vendor verification effort	Verify the integrated system and its interfaces
Reuse an interface or subsystem	Match protocols, widths, clocks, resets and timing

Trap: verified blocks do not automatically make a verified SoC. A reset mismatch, unsafe clock crossing or incorrect handshake can break the integrated system while each block works in its original environment.

5. Behavioural synthesis: what it actually builds

Behavioural synthesis, also called **high-level synthesis (HLS)**, constructs a timed RTL architecture from an algorithmic description and implementation constraints. Typical supported inputs are algorithmic C/C++ or another tool-supported high-level language. The result contains a datapath, storage, control and interfaces. HLS generates hardware; it does not simply execute the program on a CPU. [5,7]

For `y = (a+b)+c`, the arithmetic requirement alone does not say whether to use one shared adder, two chained adders or two pipeline stages. The tool must make architectural decisions:

Decision	Meaning	Example
Allocation	Choose resource types and quantities	One adder or two
Scheduling	Assign operations to time steps	First addition in cycle 1, second in cycle 2
Binding	Associate operations and values with actual resources/storage	Both additions use adder U1 in different cycles
Control generation	Sequence operations and mark valid transfers	Select operands and assert output-valid at completion
Interface generation	Connect the architecture to its environment	Define input acceptance and output handshakes

Scheduling respects data dependencies, clock requirements, resource availability and directives. Binding and scheduling influence each other because different resources can have different latency and throughput. These are related architectural choices, not merely source-code renaming. [7]

Constraints matter: clock period, resource limits, latency, initiation interval, memory/interface bandwidth and power or area goals can alter the result. An unrestricted source-language feature is not automatically synthesizable. A finite circuit needs a supported implementation model.

Boundary: HLS produces RTL architecture. Logic synthesis subsequently maps RTL toward gates; physical design places and routes the implementation. HLS does not directly place transistor polygons or finish manufacture.

6. HLS advantages, limitations and architecture trade-offs

Advantages. A concise algorithmic model can reduce manual RTL effort. Changing constraints or directives can explore different architectures and resource/performance trade-offs. A high-level model also supports fast functional testing and reuse of suitable algorithms. These are productivity opportunities; results still depend on the source structure, tool and target. [8]

Limitations. Supported language subsets exclude some host-software operations. For example, AMD's documented hardware synthesis flow does not synthesize ordinary operating-system file operations or dynamic creation/freeing of hardware memory. A testbench can still use host-side operations. Memory ports, dependencies, numeric widths and interfaces constrain parallelism. Generated RTL still needs functional, protocol and implementation verification. Do not assume globally optimal PPA or automatic clock-domain safety. [7,9]

Architecture for (a+b)+c	Resources and schedule	Main trade-off
Chained	Two adders between input and output registers	One-cycle arithmetic, but two adder delays in the path
Shared	One adder reused across two steps	Saves an adder; needs muxing, stored operands and control
Pipelined	Two adders separated by a register	Shorter stage paths; increased cycle latency and extra registers

For the simple shared schedule, two additions require two adder uses. If that adder accepts one operation per cycle, the long-run initiation interval is at least two cycles per transaction. Pipelining cannot create an extra arithmetic operation per cycle from a resource that cannot supply it.

Keep transaction ownership. If the first step stores a+b, the later step must use c from the same accepted input tuple. Either capture c or require it to remain stable under a documented handshake. In a pipeline, delay c to match its partial sum. Reset and valid signals must establish which result belongs to which request.

What to compare: use latency in time, steady-state throughput, resource/area use, power and the interface contract. Fewer cycles do not necessarily mean less time if the required clock period is longer.

7. Latency, throughput and timing worked examples

Latency is elapsed time from a defined accepted input to its corresponding valid output. **Initiation interval (II)** is spacing between accepted independent transactions. At period T, cycle latency L gives time latency LT; steady-state throughput is ideally $1/(II\,T)$ for one item per initiation. Stalls and bandwidth can lower it. [5,7]

Example A: compare the right quantities. Architecture A takes 2 cycles at 12 ns, with II=1. Its latency is 24 ns and ideal throughput is about 83.3 million items/s. Architecture B takes 1 cycle at 20 ns, with II=1. Its latency is 20 ns and throughput is 50 million items/s. B has lower single-item latency; A has greater steady-state throughput.

Example B: pipeline completion time. A pipeline has latency 3 cycles and II=1. Five independent items accepted on consecutive opportunities finish after $3 + (5 - 1) \times 1 = 7$ cycles from the first acceptance under this cycle-count convention. State the acceptance/capture convention when interpreting a waveform.

For a simple register-to-register setup budget with zero skew and uncertainty:

$$T \geq t_{CQ,\,max} + d_{comb,\,max} + t_{setup}.$$

With clock-to-Q 0.8 ns, combinational delay 5.1 ns and setup time 0.6 ns, the required period is at least 6.5 ns, corresponding to about 153.8 MHz in this model. The 5.1 ns combinational delay alone is not the full clock budget. This introduces the timing meaning used in the Week 1 HLS lecture; detailed STA comes later.

Self-check, without looking back: explain why sharing needs control; distinguish scheduling from binding; explain how II differs from latency; state why software may be the better choice for changing functionality. These prompts do not reveal the assignment choices.

8. Unix quick revision and source trail

Need	Command or concept	Detail to retain
Locate work	`pwd`, `cd`, `ls -a`	Relative paths depend on the current directory
Create/copy/move	`mkdir`, `touch`, `cp`, `mv`	touch normally preserves existing contents
Read/help	`cat`, `man`, `which`, `type`	A shell built-in differs from an external executable
Storage	`du -h`, `df -h`	Directory usage differs from filesystem free space
Processes/jobs	`ps`, `top`, `jobs`	PID and shell job number are different identifiers
Job control	Ctrl+Z, `bg`, `fg`, trailing `&`	Suspended means stopped; background can still execute

Quote paths with spaces. Linux names are generally case-sensitive. Background execution alone does not guarantee survival after terminal closure. `sudo` changes privileges; it does not fix a wrong path or syntax for a different interpreter. [6]

Sources and further reading

1. Official NPTEL course and weekly outline.

2. Lecture: IC concepts I.

3. Lecture: IC concepts II.

4. Lecture: design flow I.

5. Lecture: design flow II.

6. Unix tutorial; GNU Bash job control.

7. AMD UG1399: scheduling and binding.

8. AMD UG1399: HLS benefits.

9. AMD UG1399: C/C++ synthesis support.

The 600 practice MCQs are newly authored study questions, not verified past examination questions. A student exam discussion reports assignment-style theory and numerical questions; this is anecdotal and supplies no authenticated Week 1 exam paper. It motivates a mixed practice format, not a prediction.

Full lecture figures and original handwritten notes remain in the Day 01 reading PDF. Original assignments are taken from Kapil's enrolled 2026 NPTEL course; the practice bank is separate.

Week 2 - Revision

Logic synthesis, physical design, verification, test and post-GDS processes

Course Lessons 06-09 plus the Tcl tutorial (Lesson 10). This guide combines your new handwritten points with the remaining Week 2 material. The nine-page revision is followed by four pages of your Week 2 handwriting. Week 1's behavioural synthesis guide remains separate. [1-6]

1. Synthesis | 2. Terms | 3. Planning | 4. Clocks/routes | 5. Verification | 6. Test/yield | 7. After GDS | 8. Tcl | 9. Recall/sources | Your handwriting

1. Logic synthesis: RTL to a cell netlist

Logic synthesis translates synthesizable RTL into an equivalent circuit made from available library cells, while optimizing power, performance and area under constraints. It produces a **netlist**: instances plus their connections. Physical design subsequently determines placement and routing. Fabrication comes later. [2]

Input	What it supplies	What it does not supply
RTL, commonly Verilog/VHDL	Clocked state updates and combinational behaviour	Final cell coordinates or wire geometry
Cell library, commonly Liberty `.lib`	Cell functions, timing, power and area models	The required behaviour of the top design
Constraints, commonly SDC	Clocks, I/O timing, loads and design intent	A guarantee that the requirements are achievable

SDC means Synopsys Design Constraints. Its commands use Tcl syntax in an EDA environment; an SDC command is not automatically a standalone Tcl command. Different RTL and structural netlists may both use the `.v` extension: inspect the contents to identify the abstraction.

Three synthesis levels: behavioural synthesis starts with an algorithm and chooses a timed RTL architecture; RTL synthesis infers generic operators and storage; technology mapping chooses actual cells. Tools often combine several of these operations in one synthesis flow. Your notes' two synthesis diagrams refer to different starting abstractions.

Read the result: a conditional choice feeding an edge-triggered register can become selection logic followed by a flip-flop. The mapper may choose one mux cell or an equivalent gate network. A cell library change, tighter clock requirement or different output load can change that network even when the RTL stays the same.

Check yourself: can a legal netlist still miss timing? Yes: functional equivalence and constraint satisfaction are separate requirements. A synthesis report based on estimated wires is also not final routed timing signoff.

2. Elaboration, optimization and circuit vocabulary

Synthesis sequence: parse and elaborate RTL; infer and simplify generic hardware; map to the target library; optimize the mapped circuit. Elaboration resolves hierarchy, parameters, widths and connectivity. Typical optimization propagates constants, removes unused logic and restructures Boolean expressions while preserving the required behaviour. Mapping and optimization can iterate. [2]

Generic logic describes function without committing to a particular characterized implementation. There is no single universal generic gate set. A generic AND and a chosen standard-cell AND have the same intended Boolean function, but the latter has specific area, drive and timing properties.

Term	Concrete interpretation
Library cell	Reusable type, for example `AN2`
Library pin	Pin definition on that type, for example `AN2/A`
Instance	One use of a type: `AN2 U1` and `AN2 U2` are two instances
Instance pin	`U1/A` and `U2/A` are different connection points
Port	Interface signal on a module/design boundary
Net	Connection among pins and/or ports; it is not itself a gate
Fanout	Number of connected loads, under the stated counting convention

The slash is explanatory hierarchy notation; actual tool naming can differ. Connecting `U1/Y` to `U2/A` requires a net. Two pins both called `A` do not connect merely because they share a local pin name.

Why cell choice matters: a stronger cell can improve a loaded output's transition while adding area, input capacitance or power. Delay depends on the arc, input slew, load and operating conditions; it is not one universal number per gate. **Slew** measures edge transition time; **propagation delay** measures the response between specified input/output crossings.

FSM boundary: a finite-state machine becomes state storage, next-state logic and output logic. Detailed state minimization belongs to a later course week. The unrelated FSM exercises at the beginning of your uploaded file are preserved in its complete source copy; they are not presented as Week 2 requirements.

3. Physical design: chip planning and placement

Physical design turns logical connectivity into manufacturable geometry. A useful introductory sequence is **chip planning -> placement -> CTS -> global routing -> detailed routing -> extraction, checks and ECO iteration**. Actual flows can revisit earlier stages. [3]

Stage	Main decision	Distinction to remember
Partitioning	Divide a large design into interacting blocks	Communication and hierarchy affect the result
Floorplanning	Set die/core size, macro/I/O locations and standard-cell regions/rows	Establishes the large-scale organization
Power planning	Build supply and ground distribution, or PDN	Supports voltage integrity and reliability
Placement	Choose legal locations for individual standard-cell instances	Optimizes estimated wire length, timing and routability

Your chip-planning note about standard-cell locations means allocating regions and rows. Detailed coordinates for individual cells belong to placement. A fixed hard macro cannot simply be reshaped like a flexible block region.

PDN means power delivery network. In a simple DC segment, voltage drop is $\Delta V = IR$. With 20 mA through 0.5 ohm, the drop is 10 mV. A real network has distributed loads and transient effects. More supply wiring can improve delivery while consuming routing resources.

Utilization: state the denominator. If $U = A_{cells}/A_{usable}$, then 0.60 mm^2 of cells at 60% utilization needs 1.00 mm^2 of usable placement area. That is not automatically the whole die area: macros, margins, blockages and reserved regions matter.

Congestion is local routing demand relative to available routing capacity. Dense or poorly arranged placement can create a shortage even at moderate average utilization. Before routing, wire length and delay are estimated; placement can still optimize them. Short estimated wire length alone does not prove that all pins can be routed legally.

4. Clocks, routing, file views and ECO

Clock tree synthesis (CTS) creates a clock distribution network using buffers/inverters and routing structures. It manages sink latency, skew, load and transitions. **Latency** is arrival delay from a chosen clock reference to a sink. **Skew** is the arrival difference between specified sinks. Equal 1 ns arrival delays give zero skew, not zero latency. [3]

With sink arrivals of 10 ps and 15 ps, the arrival difference is 5 ps. Equal wire lengths alone do not ensure equal delays: cell delay, electrical loading and parasitics also matter. A gated clock needs a glitch-safe structure; a changing enable on a naive AND gate can create an unintended clock edge.

Global routing plans routes through regions and allocates coarse routing resources, often by layer. **Detailed routing** selects exact legal wires and vias, respecting spacing, width, pin-access and other rules. A promising global route can still encounter detailed-routing violations.

View/file	Primary purpose
Liberty `.lib`	Functional and characterized timing/power cell data
LEF	Abstract physical dimensions, pins, obstructions; technology routing data
Netlist `.v`	Instances and logical connectivity
SDC	Clock and timing/design intent constraints
DEF / implementation database	Placed/routed design state and related implementation information
SPEF	Modeled/extracted interconnect parasitic resistance and capacitance
GDSII	Detailed hierarchical layout geometry for release/manufacturing preparation

LEF is Library Exchange Format. An abstract pin shape does not replace a timing model, and a GDS layout does not by itself state every timing requirement.

ECO means engineering change order: a controlled implementation change, such as a local buffer, resizing or connectivity correction. It can affect timing, power, area and verification, so relevant checks must be repeated. Extraction supplies parasitics for more realistic timing/electrical analysis. Finishing routing is not the same as finishing signoff.

5. Verification: choose the right evidence

Design verification asks whether the design satisfies its specification. **Manufacturing test** screens fabricated parts for defects and performance requirements. Design-for-test hardware is planned during design; that does not make simulation a physical chip test. [4]

Method/check	What it establishes within its model	Main limitation or distinction
Simulation	Responses for applied stimuli and modeled timing	Passing selected cases is not exhaustive correctness
Model checking	Whether specified properties hold over the modeled behaviours	Depends on properties, assumptions and tractability
Equivalence checking	Agreement between two representations under the defined comparison	Both can agree while implementing a wrong specification
Static timing analysis (STA)	Timing checks over constrained paths and delay models	Needs correct clocks, exceptions, corners and constraints
Design rule check (DRC)	Layout compliance with supplied geometric/process rules	Does not prove intended circuit connectivity or function
Layout versus schematic (LVS)	Extracted layout connectivity/devices match the reference	Does not establish all timing or design-rule requirements
Electrical rule check (ERC)	Specified electrical connection/voltage rules	Rule set and tool determine the checks included

CEC means combinational equivalence checking. Formal methods include model checking and equivalence checking; those are not synonyms. The exact sequential comparison depends on the selected method and assumptions.

STA vocabulary: sequentially adjacent registers are connected by a path containing combinational logic/interconnect without intervening state elements. They need not be physically adjacent. Setup checks late arrival; hold checks early change.

For a simple single-cycle path with zero skew and uncertainty, $T \geq t_{CQ,\,max} + d_{comb,\,max} + t_{setup}$. With 0.08, 0.62 and 0.10 ns respectively, the minimum modeled period is 0.80 ns, or 1.25 GHz. Increasing the period can help setup; it does not generally cure hold. Detailed STA is taught later.

6. Test, yield, fault coverage and escapes

A **defect** is a physical imperfection; a **fault model** represents a possible erroneous circuit behaviour; a **failure** is an observed violation. A tester applies patterns and compares measured and expected responses at defined conditions. **ATE** means automatic test equipment. Wafer probing provides electrical access before packaging. [4]

Activate and observe: to detect an AND-gate input stuck at 0, drive both inputs to 1. The good output becomes 1 and the faulty output 0. Driving the other input to 0 masks the difference. Testability depends on controllability and observability, not just on applying many patterns.

Fault coverage is detected modeled faults divided by the stated modeled-fault population. Specify whether the denominator excludes untestable faults. **Yield** is the fraction of manufactured units meeting the stated goodness criteria. Neither percentage alone determines the defective fraction among shipped passing parts.

Your yield-model note is written as:

$$Y = \left(1 + \frac{Ad}{\alpha}\right)^{-\alpha}, \qquad A, d \geq 0, \quad \alpha > 0.$$

A is the modeled die/critical area, d the matching relevant defect density and α the clustering parameter. Area and density must use reciprocal units so Ad is dimensionless. For fixed d, α, increasing A lowers this model's yield. For $Ad = 1$, $\alpha = 1$ gives 0.5 and $\alpha = 2$ gives about 0.444; the large-α limit is e^{-Ad}, about 0.368. Stronger clustering (smaller α here) can leave more dies untouched at fixed mean defect density. This is a model, not a universal prediction for every process. [7]

Your 100-chip example, with its extra assumption made explicit: 90 chips are good; suppose a test detects half of the 10 bad chips and rejects no good chips. Five bad chips escape, 95 pass, and the defective fraction among passing chips is $5/95 \approx 5.26\%$, or 52,632 ppm. The assumed 50% bad-chip detection probability is not automatically the same as 50% modeled fault coverage. Testing screens parts; without repair it does not turn bad silicon into good silicon.

7. After GDS: masks, fabrication and packaged parts

Release route: layout data preparation -> mask manufacture -> wafer fabrication -> wafer test -> dicing -> assembly/packaging -> final test and classification. Screening and test order can vary by product. GDS release is a design milestone, not a completed packaged chip. [5]

Fracturing converts layout polygons into shapes supported by a mask writer. In the lecture's conventional transmissive-mask example, a pattern is written into resist on a chromium-coated transparent blank, developed, transferred by etching and inspected. Mask writing manufactures the mask; wafer exposure later uses the mask to pattern wafer resist.

Concept	Mechanism and purpose	Cost or limitation
Optical proximity correction (OPC)	Precompensate mask geometry for predicted printing distortion; e.g. line ends/corners	Process-dependent correction and mask complexity
Double/multiple patterning	Decompose a dense target into separate patterning groups/steps	Extra processing and alignment/overlay requirements
FEOL	Front end of line: forms device structures	Must meet device/process requirements
BEOL	Back end of line: builds much of the interconnect stack	Wire/via parasitics and reliability remain significant

OPC and multiple patterning can be used together. A modified mask corner need not be the desired final wafer corner. Your 193 nm note refers to a DUV lithography technology, not to every process or every minimum feature width.

Packaging supplies external connections, mechanical protection and a thermal path; it also adds electrical parasitics. DIP is **dual in-line package**; BGA is **ball grid array**. Wafer test avoids packaging some failing dies; final test catches faults introduced or revealed by later processing.

Burn-in is specified electrical/thermal stress used where appropriate to screen early-life weaknesses. **Binning** classifies parts by measured speed, power, leakage or functional capability. On-chip delay-measurement circuits may help speed classification, but they are not the definition of binning. A lower speed bin can still meet its own specification completely. Binning is not a resolution-enhancement technique.

8. Tcl: read substitutions before predicting output

Tcl = Tool Command Language. A script contains commands; the first word names the command and the other words are its arguments. Whitespace separates words; newline or semicolon separates commands when not grouped. An EDA tool embeds Tcl and adds its own commands. [6,8]

Syntax	Meaning at the ordinary Tcl parsing stage
`set n 3`	Assign 3 to variable `n`; `set n` retrieves it
`$n`	Substitute a variable's value
`[expr {1+2}]`	Run the enclosed command and substitute its result
`"value=$n"`	Group one word while allowing substitutions
`{value=$n}`	Group one word; suppress ordinary substitutions at this stage
`expr {$n + 2}`	Expression evaluator computes arithmetic; braces defer its evaluation

```tcl
set n 3
puts "n=$n"              ;# prints n=3
puts {$n [expr {1+2}]}  ;# prints $n [expr {1+2}] literally
puts [expr {$n+2}]      ;# prints 5
set values {1 2 3}
set total 0
foreach x $values {
    incr total $x
}
puts $total            ;# prints 6
```

Lists: `llength` counts elements, `lindex` reads a zero-based element, `lset` updates an element of a named list variable and `lappend` adds elements. Use `list` to preserve argument boundaries, including paths containing spaces. A Tcl array is a key-indexed variable collection; it is not the same data structure as a list.

Control and files: `if`, `for`, `while` and `foreach` control execution; `proc` defines a procedure and `return` returns a result. `source script.tcl` evaluates a script. `open` returns a channel; `gets`/`read`, `puts` and `close` use it. `exec` runs an external program. `get_cells` or `create_clock` require the appropriate EDA application's commands, not just plain `tclsh`.

Trap: braces do not mean "never evaluated." A command such as `expr`, `if` or `foreach` can evaluate the expression/script it receives later. A comment starts where Tcl expects a new command; the `;#` above deliberately starts a comment after a command.

9. Last-pass recall and source map

Answer these aloud before opening the practice questions:

1. Give the three main synthesis inputs and the netlist output. Distinguish HLS, RTL synthesis and mapping.

2. Identify a library cell, instance, instance pin, port and net in one small circuit.

3. Explain chip planning versus placement, and global versus detailed routing.

4. Distinguish clock latency from skew, and setup from hold.

5. Choose between simulation, model checking, equivalence, STA, DRC, LVS and ERC for a stated problem.

6. Separate defect, fault, failure, yield, modeled fault coverage and escaping bad parts.

7. Explain why clustering assumptions and the shipped-population denominator matter.

8. Trace GDS to a packaged chip; compare OPC, multiple patterning, burn-in and binning.

9. Predict Tcl substitutions with braces, quotes, variables and square brackets.

Sources and deeper explanations

1. Official NPTEL weekly syllabus: Week 2's implementation, verification/test and post-GDS scope.

2. Lesson 06: logic synthesis.

3. Lesson 07: physical design.

4. Lesson 08: verification and test.

5. Lesson 09: layout to chip.

6. Lesson 10: Tcl tutorial.

7. NPTEL: yield models and data analysis: clustering and the Poisson limit.

8. Official Tcl syntax manual.

Full explanations, figures and earlier complete handwriting remain in the Day 01 reading PDF (Lesson 06) and Day 02 reading PDF (Lessons 07-10). Day 02 also starts Week 3; that later Verilog material is outside this guide.

Your new material: the following four pages reproduce the Week 2 portion of your upload (lower part of original page 3 and complete pages 4-6). The complete six-page original retains its cover and earlier FSM exercises. Read the typed clarifications when a shorthand statement is ambiguous; the original handwriting is preserved as written. No assignment answer key is printed here.

Your notes: synthesis

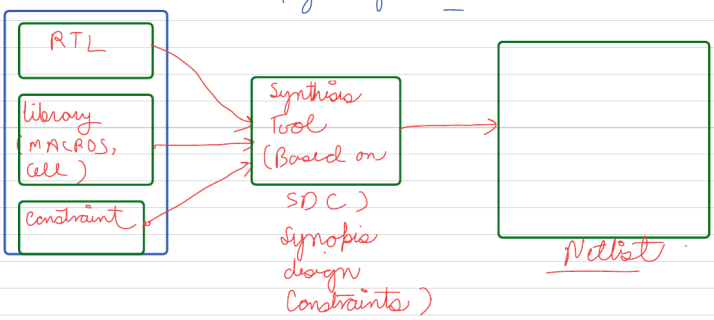
Original page 3, Week 2 portion. The earlier FSM exercise above the Week 2 heading is excluded from this view; the full source is preserved.

Week 2

behavioural Synthesis — ① Diagrams both.

logical synthesis — ② (RTL synthesis (RTL → generic logic gates))

logic Synthesis.



→ Define : pin, Port, instance pins, instance, net

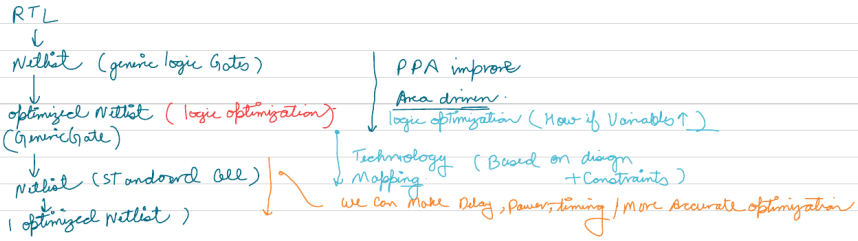
I1/A — instance, Pin A

cell inside a design is instance.

- net wire that connects diff instances & Ports.

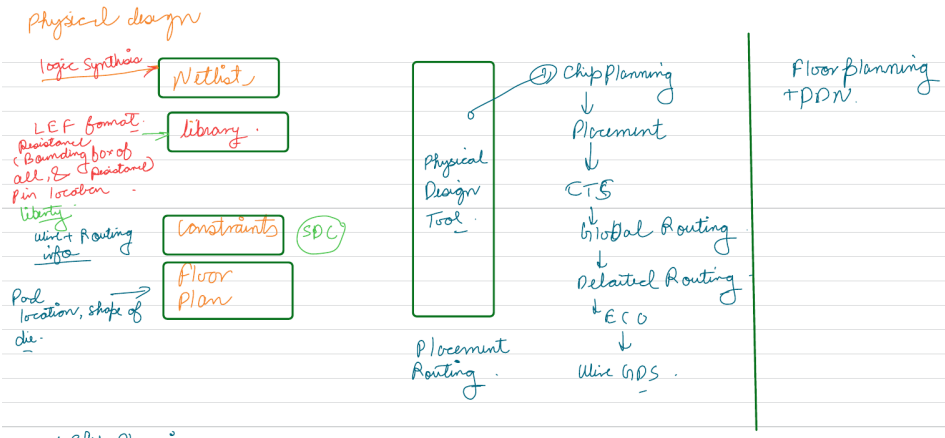
Synthesis

logical synthesis — ② (RTL synthesis (RTL → generic logic gates))



Your notes: physical design

Complete original page 4. Typed sections 3-5 expand chip planning, placement, CTS, routing and verification, and clarify their boundaries.



- Chip Planning

  - Partitioning into subsystems
  - location of standard cells
  - Allocation of area for the standard cells, macros, memory.
  - Power Planning (PDN)

Diff B/w chip Planning & Placement

2) Placement.
→ location of standard cells
— highly automated
— Wire length minimization (not have done Routing yet)
→ Timing is met ; Delay in critical path ↓
→

3) CTS.
Clock skew.

4) Routing → Global → Divide into Routing Regions.
Local — Allocating wires on each layer.

5) ECO

6)

Verification

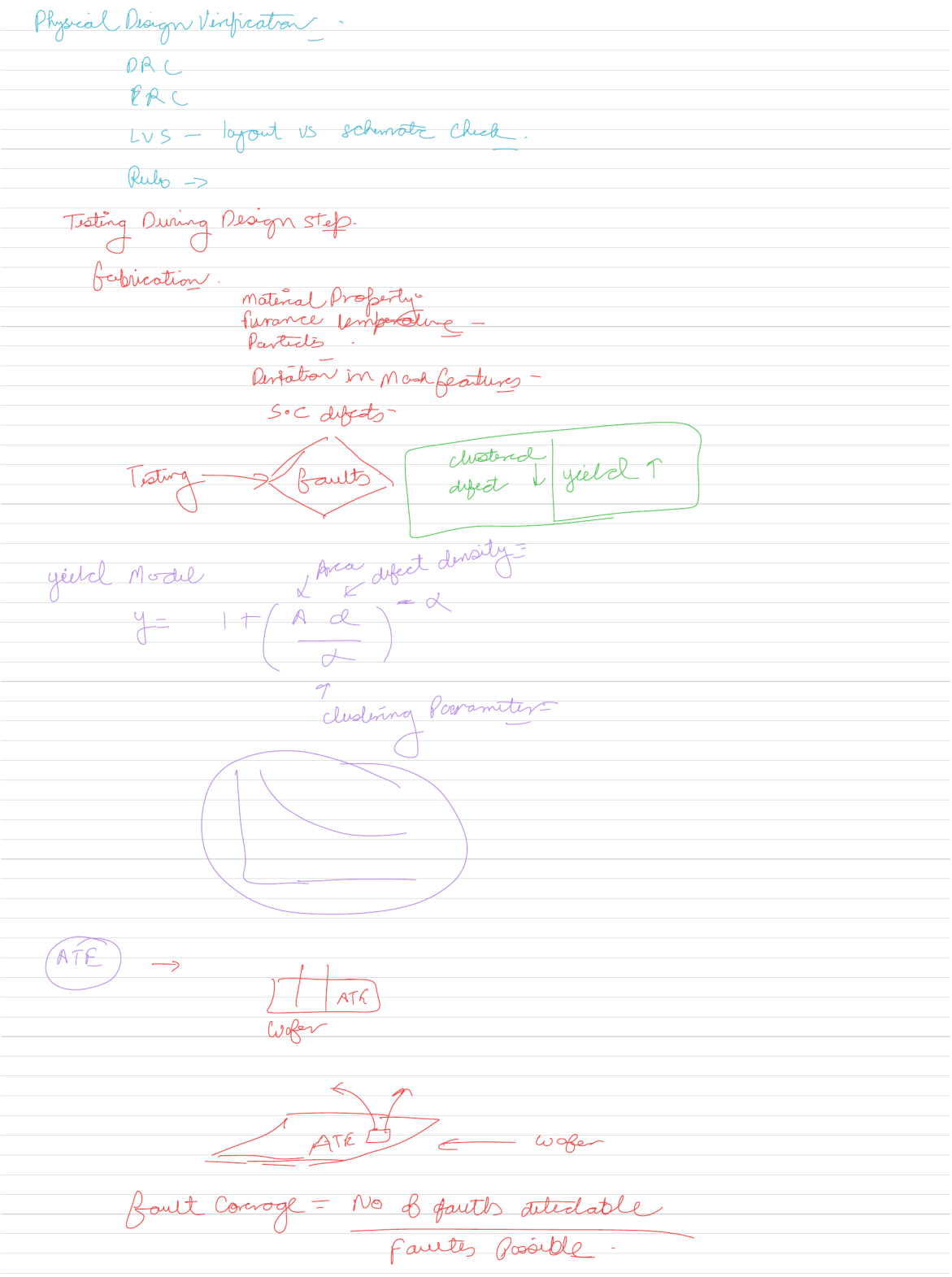
→ Simulation
(not exhaustive)

— Model Checking
formal
methods
CEC - combinational equivalence Checking
└ functional equivalent using formal methods

STA → Statically Adjacent flip flop

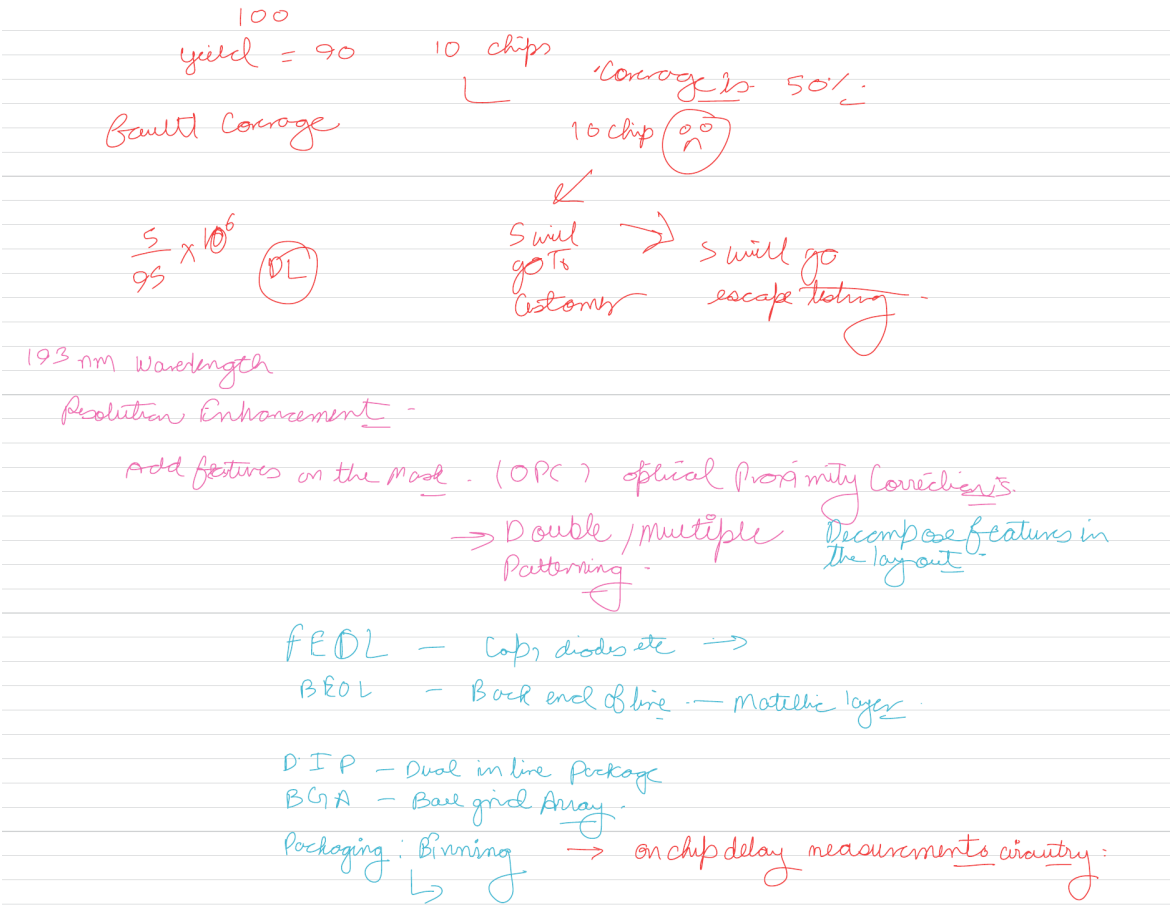
Your notes: checks, yield and ATE

Complete original page 5. Typed sections 5-6 distinguish verification from manufacturing test and explain the yield model and coverage denominator.



Your notes: escapes and manufacturing

Complete original page 6. Typed sections 6-7 make the 100-chip example's assumption explicit and expand OPC, patterning, FEOL/BEOL, packaging and binning.



Week 3 - Revision

Verilog fundamentals, simulation, coverage and the Bambu HLS tutorial

Course Lessons 11-14: Verilog I and II, functional verification using simulation, and Tutorial 3. Ten typed revision pages are followed by six original handwritten pages. Read widths and event ordering before predicting an answer; assignment choices are not marked here. [1-5]

1. Types | 2. Widths | 3. Operators | 4. Modules | 5. Time | 6. Assignments | 7. Queues | 8. Coverage | 9. HLS | 10. Recall/sources | Your handwriting

1. HDL thinking, declarations and four-state values

An **HDL** describes concurrent hardware, connections, state and timing. A simulator evaluates the model; synthesis translates a supported subset into hardware. Successful simulation does not make every construct synthesizable. A testbench delay does not specify a manufactured gate delay. Different processes are concurrent, while statements inside one `begin ... end` execute in their defined order. [2]

Declaration or value	Meaning and common misunderstanding
`wire`	A net derives its value from its drivers; an ordinary undriven wire reads `z`
`reg`	A procedural variable retains its last assigned simulation value; uninitialized it reads `x`
`wire [7:0] bus`	One eight-bit vector; `bus[3:0]` selects four bits
`reg [7:0] mem [0:15]`	Sixteen words of eight bits each; 128 data bits in total
`integer`, `time`, `real`	Traditional signed 32-bit integer, unsigned 64-bit time value, and floating-point simulation value
`0`, `1`, `x`, `z`	Known low, known high, unknown, and high impedance; these are simulation states

`reg` **does not mean flip-flop.** A complete combinational process can assign a `reg` without storing state. An edge-triggered process can describe flip-flops; an incomplete combinational assignment can require retention and infer a latch. The process and assignment paths determine the behaviour.

Unknown versus released: `x` indicates insufficiently known information, such as uninitialized state or conflicting equal-strength drivers. `z` describes a released drive contribution. A driven `1` and a released `z` can resolve to `1`; `z` is not a second spelling of `x`. For operator evaluation, controlling bits can give known results: `0 & x` is 0 and `1 | x` is 1.

Verilog is case-sensitive. Keywords are lowercase; ordinary identifiers begin with a letter or underscore. Use `//` for a line comment and `/* ... */` for a block comment. `[7:0]` and `[0:7]` are both legal ranges, but their index directions differ. Do not infer bit significance from an index number alone.

2. Literals, width, truncation and signedness

A based integer literal is **size, apostrophe, optional `s`, base, digits**, for example `8'shF6`. Size counts bits: binary, octal and hexadecimal digits do not change the declared width. Underscores are readability separators. A leading minus is a unary operator, not a digit inside the literal. [2]

Example	Stored bits or result	Why
`8'h3C`	`00111100`, unsigned 60	Each hexadecimal digit supplies four bits
`6'hA5`	`100101`, unsigned 37	Six bits retain the least significant six of `10100101`
`8'b101`	`00000101`	Ordinary positive digits are zero-padded on the left
`8'bz01`	`zzzzzz01`	A leading `z` pads the unfilled upper positions with `z`
`8'bx01`	`xxxxxx01`	A leading `x` similarly extends unknown positions
`8'shF6`	`11110110`, signed -10	Signed interpretation uses the top bit and two's complement
Same bits treated unsigned	246	The bits alone do not determine signed interpretation

Two's complement: at eight bits, -10 has the encoding $256 - 10 = 246$. This explains the bit pattern without confusing signed value and unsigned encoding. Narrowing discards upper bits; it is not rounding or saturation.

Preserve carry explicitly. For eight-bit unsigned inputs, the largest sum is $255 + 255 = 510$, requiring nine result bits. Write `sum = {1'b0,a} + {1'b0,b};` with a nine-bit destination. At `a=240`, `b=32`, the full sum is 272; the low eight bits alone hold 16. A wider destination cannot recover information already lost in a self-determined narrow intermediate.

Signed operations need a signed context. For signed eight-bit `s=-10`, an explicitly signed temporary assigned `s >>> 1` holds -5. `>>` shifts in zeros; `>>>` sign-extends only when the effective left operand is signed. Mixed signed/unsigned expressions can change extension and interpretation. Use deliberate signed declarations/constants and inspect intermediate widths.

Unsized decimal literals are signed and at least 32 bits in traditional Verilog. String literals use eight bits per character: five characters need 40 bits in a traditional packed-vector representation. Type, width and signedness are separate questions.

3. Operators, equality, unknowns and wildcard cases

First identify whether an operator works on individual bits, reduces one vector, or tests truth. Assume the explicitly sized binary values below. [3]

Expression	Result	Interpretation
`4'b0110 & 4'b1010`	`4'b0010`	Bitwise AND at each position
`4'b0110 && 4'b1010`	`1'b1`	Logical AND: both operands are nonzero
`&4'b1110`	`1'b0`	Reduction AND produces one bit
`^4'b1100`	`1'b0`	Reduction XOR: an even number of ones
`~4'b0110` / `!4'b0110`	`4'b1001` / `1'b0`	Bitwise complement / logical negation
`{3'b101,2'b10}`	`5'b10110`	Left item occupies the more significant portion
`{3{2'b01}}`	`6'b010101`	Replicate the two-bit pattern three times
`1'bx == 1'bx` / `1'bx === 1'bx`	`x` / `1`	Indeterminate logical equality / exact case equality

Conditional operator: with a one-bit unknown select, equal-width integral alternatives merge bit by bit. `1'bx ? 4'b1100 : 4'b1010` gives `4'b1xx0`. Equal bits survive; disagreements become unknown. A procedural `if` with an unknown one-bit condition does not take its true branch. That can hide an unknown control instead of exposing the ambiguity.

Construct	Matching rule
`case`	Exact four-state matching
`casez`	`z`/`?` positions are ignored in the selector or item
`casex`	`x`, `z` and `?` positions are ignored in the selector or item

In a based literal, `?` encodes `z`; it is not a fifth logic state or a universal wildcard. Wildcard behaviour comes from the surrounding case construct. An unintended `z` selector can broaden `casez` matches; `casex` can conceal uninitialized `x` control bits. Define intentional patterns and defaults.

Exam method: record each operand's width and signedness, apply the operator's four-state rule, then handle destination conversion. Parentheses make precedence clear but do not automatically widen operands. Unknown simulation values are not freely chosen synthesis don't-cares.

4. Modules, parameters, functions and tasks

A **module definition** is a type; an **instance** is one use with its own connections and state. `input`, `output` and `inout` describe port direction. Named connections `.a(parent_a)` connect the child's port `a` to the parent's signal `parent_a`; the two names need not match. [3]

A **parameter** is selected during elaboration. It can configure widths or structure and can be overridden per instance. It is not a changing runtime signal. `localparam` defines an internal constant that an instance cannot override. A generate condition selects structure at elaboration; a procedural `if` can describe behaviour during execution.

Verilog
```
module pass #(parameter W=4)
  (input wire [W-1:0] a, output wire [W-1:0] y);
  assign y = a;
endmodule

module top(input wire [7:0] data, output wire [7:0] out);
  pass #(.W(8)) u_pass (.a(data), .y(out));
endmodule
```

The instance has eight-bit ports because `W=8` is resolved before execution. Changing `data` does not change `W`. A continuous `assign` at module scope drives a net whenever its expression changes. It is not an ordinary statement inserted into an `always` block. Legacy procedural `assign/deassign` are a separate language facility.

Traditional Verilog rule	Function	Task
Result and use	One return value; usable in an expression	Called as a statement; output/inout arguments can return values
Arguments	At least one input; input arguments only	Zero or more input/output/inout arguments
Timing controls	Cannot suspend with `#`, `@` or `wait`	May contain timing/event controls
Calls	Other functions, not tasks	Functions or tasks

A task is permitted to take simulation time; it need not do so. A function's zero simulation time does not mean zero hardware delay or zero computer effort. Neither label alone determines synthesizability or circuit storage. SystemVerilog extends these traditional rules. Default local storage is static; `automatic` gives separate invocation storage where needed.

5. Processes, sensitivity lists, edges and time

An `initial` **block** starts once and finishes when its statements finish. It can contain waits/delays, so it need not finish at time zero. An `always` **block** repeats its body. Separate processes start concurrently; source order does not give one priority. `begin/end` sequences statements within a process; `fork/join` runs concurrent branches and waits for all branches. [3]

Sensitivity is an event condition. `@(a or b)` resumes on a change of either signal; the `or` separates events, not arbitrary gate operators. `always @(enable)` does not continually run while enable is high and does not react to every other value read in its body. Use `always @*` for ordinary combinational processes, with every output assigned on every path.

Single-bit event	Transitions that count
`posedge`	`0->1`, `0->x`, `0->z`, `x->1`, `z->1`
`negedge`	`1->0`, `1->x`, `1->z`, `x->0`, `z->0`
Neither	`x->z`, `z->x`, or no change

These are simulation rules, not a promise that an uncertain electrical clock is safe. In an edge expression that is a vector, edge detection uses its least significant bit; normal clocks should be one bit.

Verilog
```
`timescale 1ns/1ps
module clock_demo;
  reg clk = 1'b0;
  always #4 clk = ~clk;
  initial #25 $finish;
endmodule
```

After initialization, clock toggles occur at 4, 8, 12, 16, 20 and 24 ns. Four nanoseconds is the half-period; period is 8 ns and frequency is 125 MHz. An uninitialized `x` clock remains `x` when complemented. A repeating process without a delay/event control can prevent time from advancing.

Time unit versus precision: `1ns/1ps` means delay values use nanoseconds and are rounded to 1 ps resolution. `#2.3456` therefore requests 2.3456 ns and rounds to 2.346 ns. Simulation time is modeled circuit time; wall-clock runtime is how long the computer takes. A `for` loop uses a clock cycle per iteration only if its body explicitly waits for those events.

6. Blocking, nonblocking and delayed assignments

Blocking = completes the assignment before the next statement in that process. **Nonblocking** <= evaluates the RHS when reached and schedules a later LHS update; without an explicit delay, that update belongs to the NBA region of the current time slot. It does not automatically wait for the next clock or the end of the entire simulation. [3,4]

For a two-stage clocked pipeline, assume known reset values, inputs stable before each rising edge, and observations after NBA updates:

```
Verilog
always @(posedge clk) begin
  q1 <= d;
  q2 <= q1;
end
```

Edge	Input `d`	Old `q1`	New `q1`	New `q2`
First, after reset	12	0	12	0
Second	7	12	7	12
Third	31	7	31	7

Both RHS values are sampled before these pending updates. Replacing both assignments with = in the same order lets `q2` see the newly assigned `q1`, changing the behaviour. Separate blocking clocked processes can also race. Use blocking assignments for ordinary combinational work and nonblocking assignments for clocked state, while checking the whole process.

Delayed assignments: execute each sequence from time zero, with constant RHS values and a 1 ns time unit:

Sequence inside one process	Times when the destinations become one
`a = #3 1; b = #8 1; c = #5 1;`	`a`: 3 ns; `b`: 11 ns; `c`: 16 ns
`p <= #3 1; q <= #8 1; r <= #5 1;`	`p`: 3 ns; `q`: 8 ns; `r`: 5 ns

The blocking waits accumulate. The nonblocking writes are all scheduled while the process runs at time zero. With `q <= #8 d`, sample `d` now and delay the write. With `#8 q <= d`, wait first and sample `d` later. If `d` changes from 0 to 1 at 4 ns, the two forms write 0 and 1 respectively at 8 ns. These explicit delays are simulation models, not synthesizable arbitrary delay elements.

7. Simulation queues, observations and races

Event-driven simulation evaluates activated processes and applies updates, which can activate more processes. It settles eligible work at a timestamp before moving to future scheduled times. Several event iterations can occur without time advancing. The lecture's time-wheel example (`T % m`) illustrates an implementation data structure, not a mandatory simulator architecture. [4]

Classic Verilog region/category	Typical activity
Active	Ordinary process execution, blocking updates, continuous-assignment evaluation, NBA RHS evaluation
Inactive	Continuation after procedural `#0`
NBA update	Apply previously scheduled nonblocking LHS values
Monitor	Settled observations by `$strobe` and `$monitor`
Future events	Become eligible at later simulation times

An NBA update can awaken more active work at the same time. This is not a one-pass queue diagram. The table is the classic Verilog model used in the course; SystemVerilog has additional scheduling regions.

Verilog
```
`timescale 1ns/1ps
module queue_check;
  reg a;
  initial begin
    a = 0;
    a <= 1;
    $display("active a=%b", a);
    $strobe("settled a=%b", a);
    #1 $finish;
  end
endmodule
```

The display prints `active a=0`; the strobe prints `settled a=1`. Both belong to time zero. `$display` observes when executed; `$strobe` waits for settled end-of-slot observation. `$monitor` reports changes to its listed values. A procedural `#0` is still before NBA updates and is not a universal way to sample new clocked state.

A **race** makes the result depend on an unspecified ordering of eligible processes. Driving an input with = at the same edge that the DUT samples it can race. In a small testbench, drive on the opposite edge and check after the NBA updates; a suitable SystemVerilog clocking block is another approach. Races are unrelated to any fixed coverage percentage.

Cycle-based simulation evaluates synchronous next-state behaviour at clock boundaries and can reduce scheduling overhead for suitable models. Its abstractions must fit the design; it is not the default answer for arbitrary asynchronous or fine-delay behaviour. Neither simulation style proves an untested specification.

8. Testbenches, waveforms and coverage

The **DUT** is the design under test. The **testbench** generates stimulus, establishes clock/reset conditions and compares responses with an expected model. A **simulator** executes the design and testbench; a waveform viewer displays recorded activity. A plausible-looking trace is weaker evidence than a check of the actual requirement. [4]

Coverage or evidence	Question answered	Limitation
Statement/line coverage	Did the measured source statements execute?	Executed does not mean correct
Branch coverage	Were the control alternatives exercised?	Does not establish all data combinations
FSM state / transition coverage	Which states / which transitions were reached?	Visiting endpoint states does not prove a particular transition occurred
Toggle coverage	Did measured bits make the required transitions?	Does not show that every feature works
Functional coverage	Did specified scenarios/bins occur?	Requires an explicit, adequate coverage model
Self-checking result	Did an observed response match the expected result?	Establishes only the checks actually performed

Why 100% code coverage is insufficient: a missing feature can have no RTL line to mark uncovered. An adder can execute on every test while the checker ignores its carry output. Functional coverage is usually more manual because someone must specify meaningful scenarios and crosses. Coverage and correctness are complementary evidence.

For an enabled counter, check reset from a nonzero state, increment at each enabled edge, retention while disabled and wrap at the maximum value. A final-value-only comparison can miss a skipped increment followed by a double increment. Drive stable inputs before sampling and compare after the DUT's NBA updates have settled.

Unknown-sensitive checker: `if (actual !== expected)` treats unexpected `x` or `z` as a mismatch when the expectation is known. `!=` can itself evaluate to `x`, allowing an ordinary `if` to miss the intended failure branch.

VCD = Value Change Dump. `$dumpfile` names the output and `$dumpvars` selects recorded hierarchy. VCD stores times and values, not a pass/fail proof. `$finish` ends simulation; `$stop` requests a stop whose interactive behaviour depends on the simulator. Successful compilation/elaboration alone does not establish a passing simulation. [7]

9. Bambu: C behaviour to a timed RTL architecture

Tutorial 3 introduces **Bambu**, a high-level synthesis tool. It takes supported high-level behaviour and implementation constraints, selects resources, schedules and binds operations, and generates RTL control, datapath and interfaces. The algorithm does not uniquely specify the number of arithmetic units, cycle latency or initiation interval. [5]

The tutorial's top function illustrates ordered decisions. This compact form preserves that behaviour:

```c
long func(int j, int k, int c, int d) {
  int i;
  if (c > 2)      i = j - k;
  else if (d < 5) i = j + k;
  else           i = 12;
  return i;
}
```

Inputs with `j=9`, `k=4`	**Selected branch**	**Returned value**
`c=3`, `d=4`	First condition wins even though the second comparison is also true	5
`c=2`, `d=4`	First false, second true	13
`c=2`, `d=5`	Both false; constant branch	12

Preserve comparison boundaries and branch priority. Returning `long` does not automatically widen an earlier `int` operation: avoid signed overflow and check the tool's type interpretation. The handout's surrounding software driver uses uninitialized local inputs; initialize test data before treating software output as a numerical reference.

Architecture trade-off: one shared multiplier can save arithmetic resources but adds selection, storage and scheduling restrictions. More resources or pipelining may improve throughput at an area/power cost. Check actual generated reports rather than assuming a source line maps to one gate or one cycle.

Latency needs a defined input-acceptance event and matching valid-output event. **Initiation interval (II)** is spacing between accepted transactions. At 80 MHz with II=2, ideal steady-state throughput is 40 million transactions/s if interfaces do not stall; latency can be a different number of cycles. Generated RTL must also preserve transaction pairing and handshake behaviour.

Verification compares numerical results and protocol/cycle behaviour with defined inputs, including boundaries. Running the C program alone does not verify the generated RTL. Historical AppImage/install commands belong to the course demonstration; the revision does not claim a new Bambu installation or synthesis run.

10. Final recall and source trail

Before practice, explain these without looking back:

1. Why can a `reg` describe combinational logic? How does a `wire` obtain its value?

2. How do width and signedness alter a stored literal, shift or sum?

3. Distinguish bitwise, logical and reduction operators; then explain `==`, `===`, `casez` and `casex`.

4. Distinguish module type, instance, parameter, function and task.

5. Why can `initial` finish after time zero? Which unknown transitions count as edges?

6. Trace a two-register NBA pipeline and a delayed-RHS sample. Identify when each RHS is read.

7. Explain active, inactive and NBA work, settled observation and a testbench race.

8. Explain why full code coverage, a waveform and a compiled design are not equivalent to specification correctness.

9. Identify Bambu's input/output abstractions and separate latency, II and throughput.

Sources and deeper reading

1. Official NPTEL course and weekly syllabus.

2. Lesson 11: Introduction to Verilog I.

3. Lesson 12: Introduction to Verilog II.

4. Lesson 13: Functional verification using simulation.

5. Lesson 14: Bambu HLS tutorial.

6. Sutherland's Verilog-2001 reference guide: traditional language rules; SystemVerilog extends several of them.

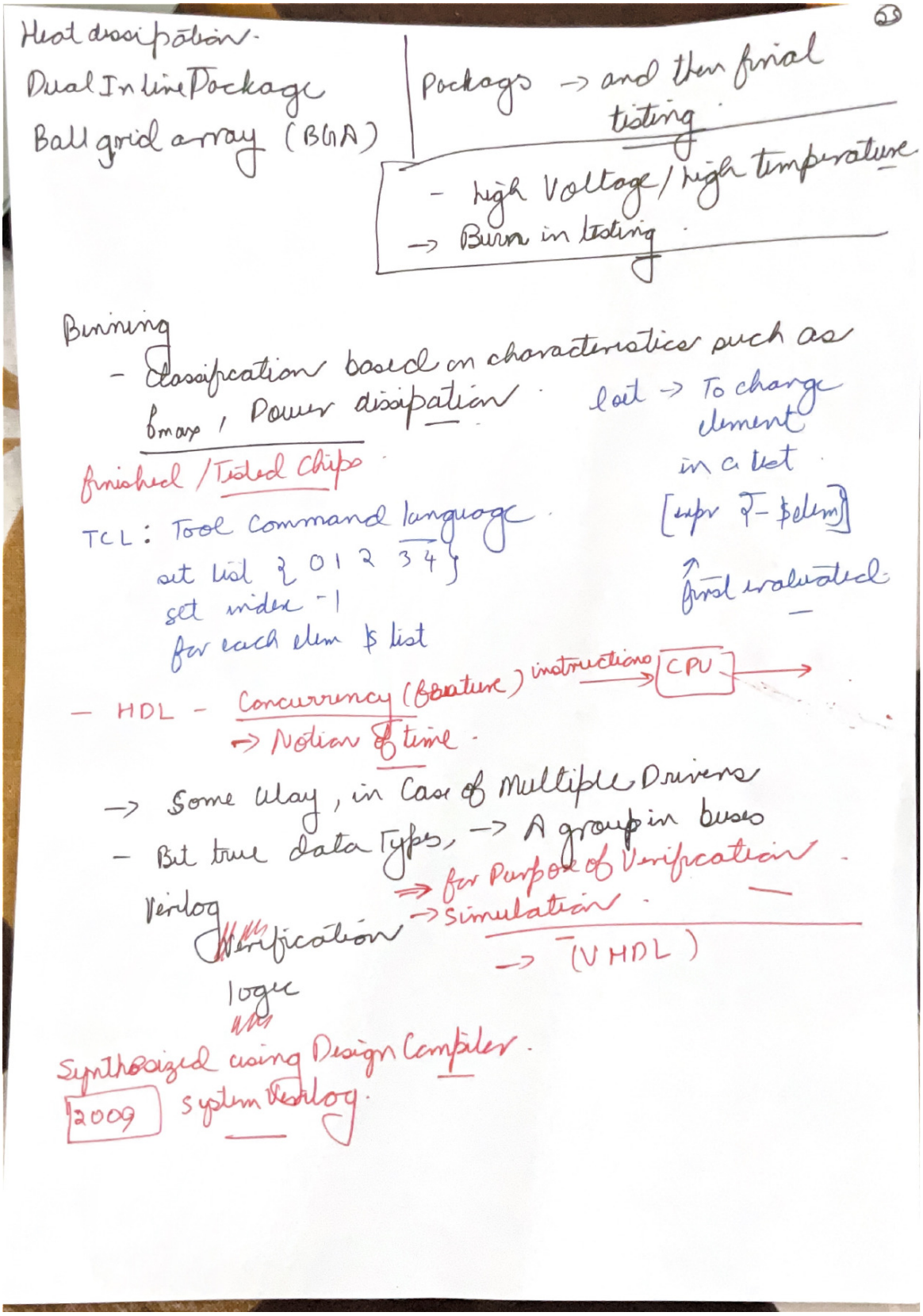
7. Official Icarus simulation documentation: compilation, elaboration and simulation workflow.

Handwriting map: the next six pages preserve Part 2 original pages 2-4 and Scan A pages 1-3. Part 2 page 2 also retains earlier Week 2 notes; its HDL section begins the Week 3 material. The typed explanations clarify shorthand about `reg`, unknowns/wildcards, functions/tasks and NBA timing. No separate handwritten Bambu page is present in the collection.

Full lecture explanations and figures remain in the Day 02 reading PDF (Lessons 11-12) and Day 03 reading PDF (Lessons 13-14). Later Day 03 synthesis/optimization lessons belong to Week 4. The handwritten index and editable language examples provide the detailed source trail.

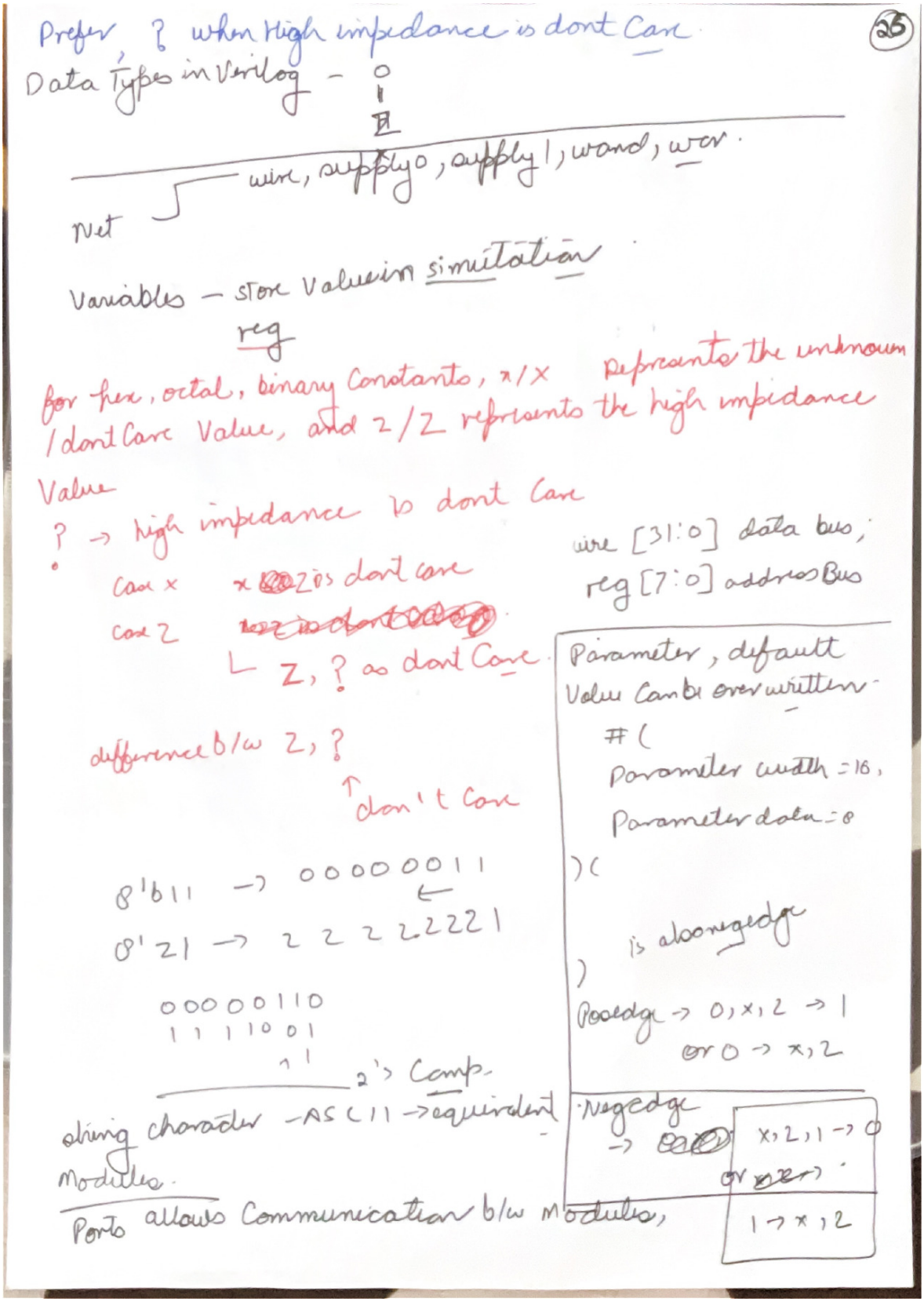
Your notes: HDL foundations

Complete Part 2 original page 2. The lower HDL section introduces Week 3 concurrency, time, multiple drivers and bit-accurate data; the earlier package/Tcl material belongs to Week 2. See typed section 1.



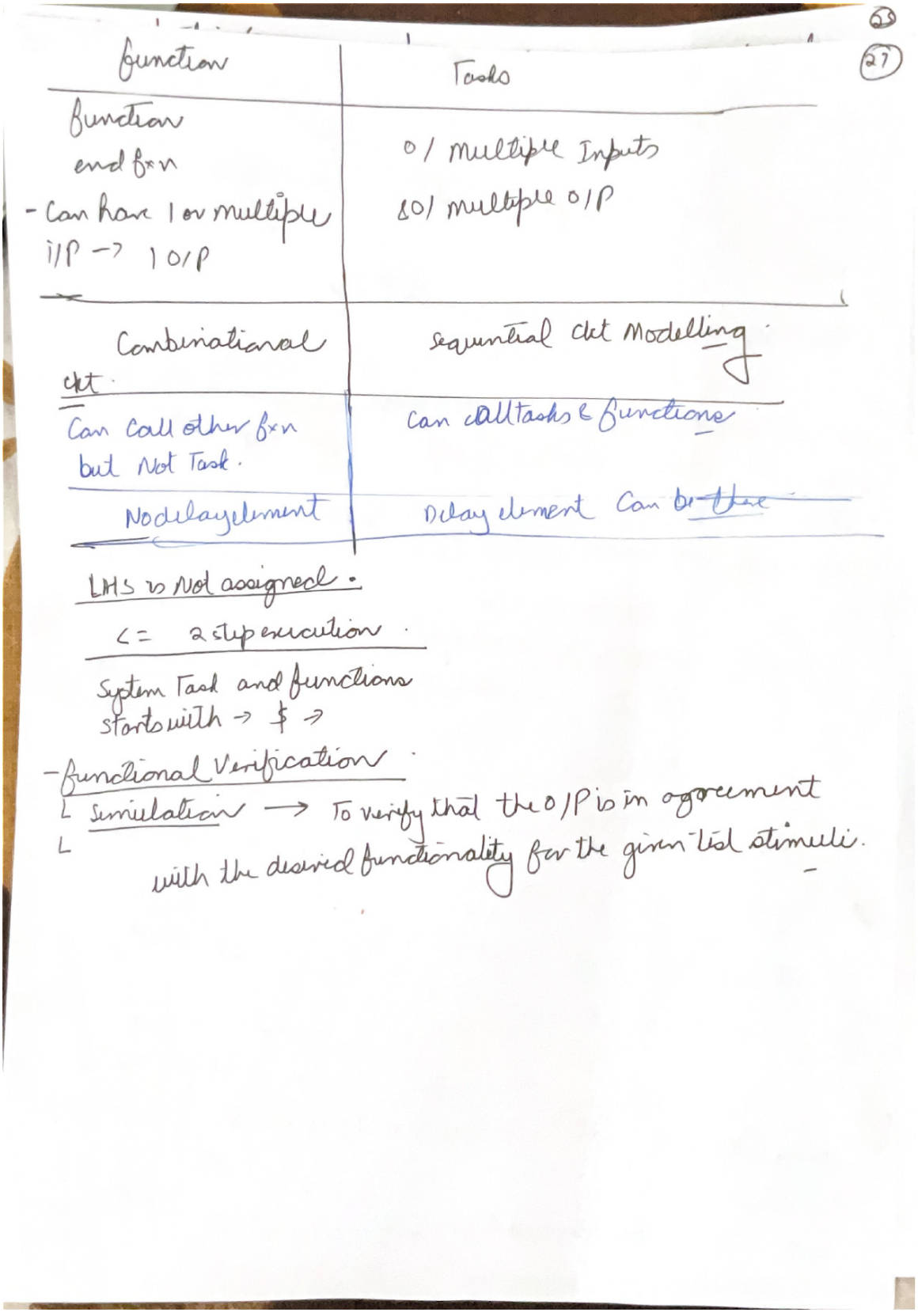
Your notes: values, types and events

Complete Part 2 original page 3. Typed sections 1-5 explain nets/variables, unknowns and wildcards, sized constants, parameters, strings, modules and four-state edge events.



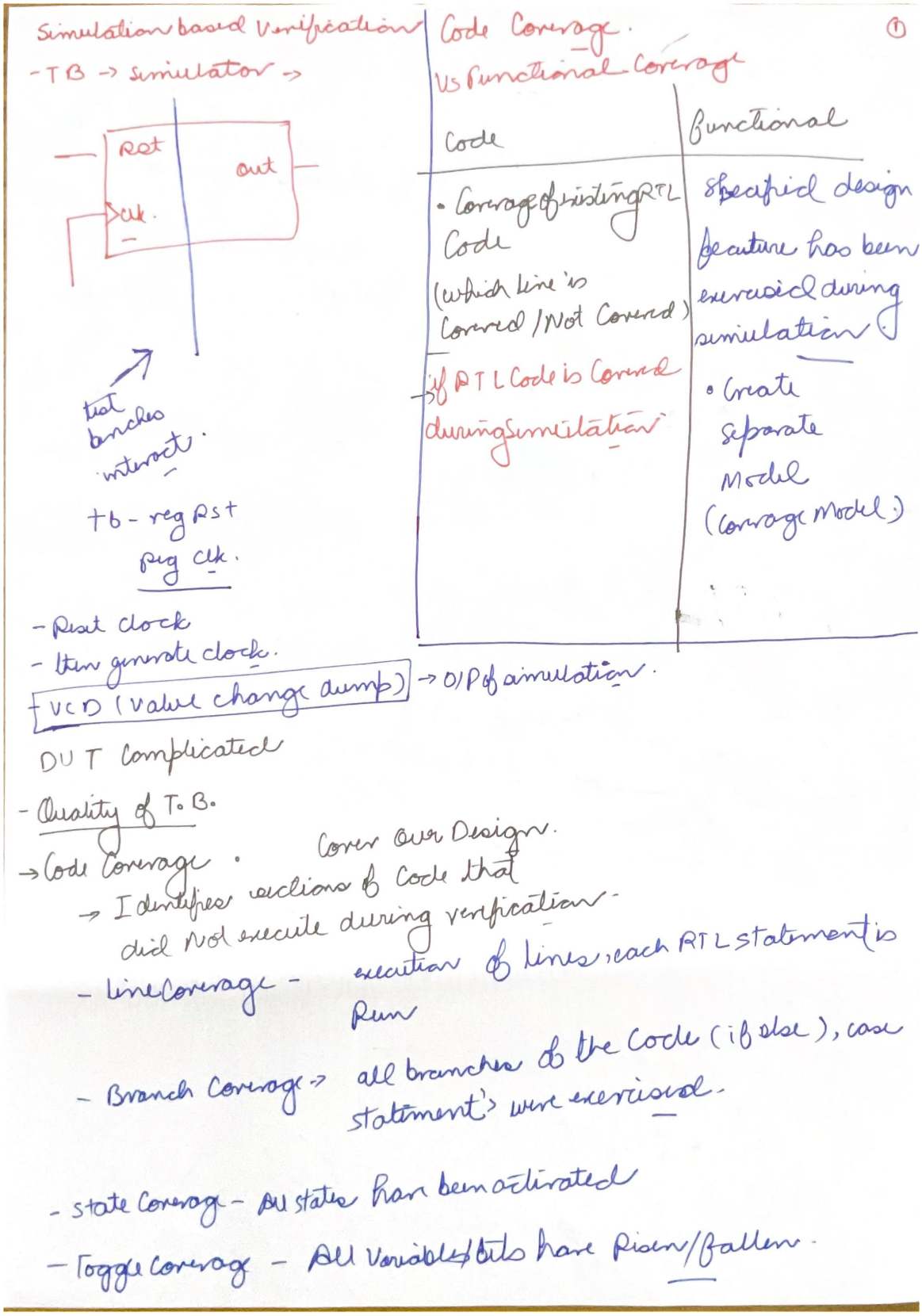
Your notes: functions and assignments

Complete Part 2 original page 4. Typed sections 4, 6 and 8 clarify traditional function/task timing, RHS evaluation versus NBA updates, system tasks and functional verification.



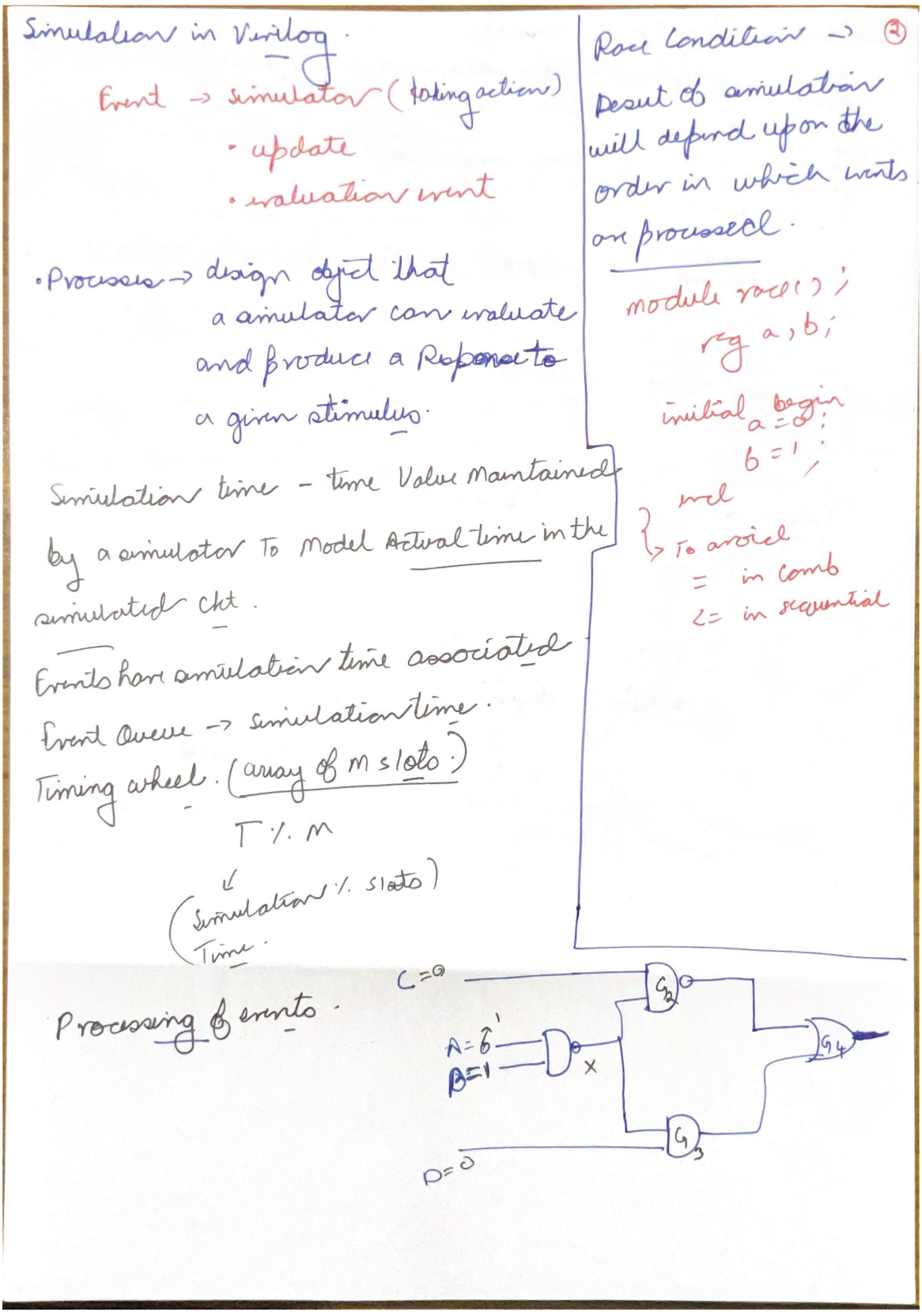
Your notes: testbench and coverage

Complete Scan A original page 1. Typed section 8 explains DUT, clock/reset stimulus, VCD, line/branch/state/toggle coverage and a separate functional coverage model.



Your notes: events and races

Complete Scan A original page 2. Typed sections 6-7 explain evaluation/update events, simulation time, the illustrative time wheel, event propagation and race conditions.



Your notes: stratified event queue

Complete Scan A original page 3. Typed section 7 traces the classic Verilog event regions and why display and settled observations can see different values at one timestamp.

